

PSTAT100 Data Science Concepts and Analysis

Lab 2: Tidy Data and Data Transformations

AUTHOR
Yuting Ma

Name: _____ Section: _____

Objectives

This lab covers the following topics:

- **Tidy datasets:** identifying variables and observational units.
- **Reshaping tables:** `pivot_longer()` and `pivot_wider()`.
- **Selecting data:** `filter()`, `select()`, and `slice()`.
- **Grouped summaries:** `group_by()` and `summarize()`.

Student worksheet

1. Tidy Datasets

1.1 Creating the Example Dataset

These are fictional scores out of 10. Each student belongs to section A or B; **NA** means the score was not recorded. Our question is about scores by quiz, so we choose **one student taking one quiz** as the observational unit.

```
library(tidyverse)
scores_wide <- tibble(
  student = c("Ana", "Ben", "Cam", "Dee"),
  section = c("A", "A", "B", "B"),
  quiz1 = c(8, 6, 9, 7),
  quiz2 = c(10, NA, 8, 9)
)
```

student	section	quiz1	quiz2
Ana	A	8	10
Ben	A	6	NA
Cam	B	9	8
Dee	B	7	9

For the student-quiz unit, **quiz1** and **quiz2** are values of a quiz variable stored as column names. We want columns **student**, **section**, **quiz**, **score**. If the unit were a student with two distinct score variables, a wide layout could also be appropriate.

Hands-on Exercise 1: Recognize the unit

A table has columns **species**, **body_weight_kg**, and **brain_weight_g**, with one row per species. Is it tidy for studying brain and body weight by species? State the observational unit and explain whether you need to pivot.

2. Reshaping Tables

`pivot_longer()` moves column names into a variable. `pivot_wider()` moves values from one variable into column names. These are **tidyr** functions.

2.1 Pivoting Longer

```
scores_long <- scores_wide |>
  pivot_longer(
    cols = starts_with("quiz"),
    names_to = "quiz",
    values_to = "score"
  )
scores_long
```

Result: **8 rows, 4 columns**, in student order with quiz1 then quiz2. The first four rows are:

student	section	quiz	score
Ana	A	quiz1	8
Ana	A	quiz2	10
Ben	A	quiz1	6
Ben	A	quiz2	NA

`cols` chooses columns to reshape. `names_to` and `values_to` are quoted because they provide **new column names**. The missing score is retained.

2.2 Pivoting Wider

```
scores_long |>
  pivot_wider(
    id_cols = c(student, section),
    names_from = quiz,
    values_from = score
  )
```

Result: the original **4-row, 4-column** table in Section 1. Here `quiz` and `score` are

existing columns, so they can be unquoted. Each student-quiz combination occurs once; duplicate combinations would need attention.

Hands-on Exercise 2: Both directions

a. Rewrite the longer pivot using `cols = c(...)` instead of a helper. Use new column names `assessment` and `points`; save as `scores_alt`.

```
scores_alt <- scores_wide |>
  pivot_longer(cols = c(____, ____),
               names_to = "____", values_to = "____")
```

b. Convert `scores_alt` back to wide. How many rows will the result have?

```
scores_alt |>
  pivot_wider(id_cols = c(student, section),
              names_from = ____, values_from = ____)
```

Rows: _____

3. Selecting Rows and Columns

These **dplyr** functions choose rows or columns.

Function	Chooses	Example
<code>filter()</code>	Rows meeting a condition	<code>filter(score >= 9)</code>
<code>select()</code>	Columns by name or helper	<code>select(student, score)</code>
<code>slice()</code>	Rows by position in current order	<code>slice(1:2)</code>

Use unquoted column names and quoted text values. `==` compares values; commas in `filter()` mean AND; `|` means OR.

3.1 Filtering and Selecting

```
scores_long |>
  filter(section == "A", score >= 9) |>
  select(student, quiz, score)
```

student	quiz	score
Ana	quiz2	10

`filter()` keeps only TRUE conditions; missing comparisons are dropped. Use `is.na(score)` to find missing scores.

3.2 Slicing by Position

```
scores_wide |>
  select(student, starts_with("quiz")) |>
  slice(c(1, 3))
```

Result: Ana (quiz1 = 8, quiz2 = 10) and Cam (quiz1 = 9, quiz2 = 8).

`slice()` uses current row order, not score size; grouped slicing works within each group.

Hands-on Exercise 3: Write code and predict

the result

a. From `scores_long`, keep section B scores of at least 9; return only `student` and `quiz`. Write the code and the student-quiz pairs returned.

```
scores_long |>
  filter(_____) |>
  select(_____)
```

Returned pairs: _____

b. From `scores_wide`, keep only columns `student` and `quiz2`, then keep row positions 2 and 4. Write code. Which two students remain?

```
# Your code:
_____
```

Students: _____

4. Grouping and Summarizing

`group_by()` tells later operations which rows belong together; it does not itself reduce the number of rows. `summarize()` with one summary value per group returns one row per group. `summarise()` is an equivalent spelling.

4.1 Summarizing by Section

```
scores_long |>
  group_by(section) |>
  summarize(
    n_rows = n(),
    n_recorded = sum(!is.na(score)),
    mean_score = mean(score, na.rm = TRUE),
    .groups = "drop"
  )
```

section	n_rows	n_recorded	mean_score
A	4	3	8.00
B	4	4	8.25

`n()` counts **rows**, including missing scores. Section A's mean is $(8 + 10 + 6)/3 = 8$, not divided by 4. `na.rm = TRUE` excludes missing scores from the calculation, not from the dataset. Without it, A's mean is `NA`. `.groups = "drop"` makes the result ungrouped.

Think about the denominator: Here rows are student-quiz observations, not students. Without `group_by()`, `summarize()` would give one overall row.

Hands-on Exercise 4: Two grouping variables

Compute the **mean recorded score** separately for each section and quiz. Also report the number of recorded scores. Use `scores_long`.

```
scores_long |>
  group_by(_____) |>
  summarize(
    n_recorded = _____,
    mean_score = _____,
    .groups = "drop"
  )
```

Fill in the expected output. Then explain why A-quiz2 has only one recorded score even though section A has two students.

section	quiz	n_recorded	mean_score
A	quiz1	_____	_____
A	quiz2	_____	_____
B	quiz1	_____	_____
B	quiz2	_____	_____

Explanation: _____

5. Independent Practice

Work alone or with a partner. Use the setup on Section 1 and `scores_long` from Section 2. Write code and predict results before checking in R.

Hands-on Exercise 5: Build a complete pipeline

Starting with `scores_wide`, (1) pivot to `quiz` and `score`; (2) keep only quiz2 rows with recorded scores; (3) calculate the mean score and number of recorded scores by section; (4) keep only sections with a mean of at least 9. Call the count column `n_recorded` and the mean column `mean_score`.

Your pipeline (use the space below):

Which section(s) remain? Give the count and mean. Why is the last filter after `summarize()` rather than before it?

Hands-on Exercise 6: Diagnose and repair

A classmate wants the mean score by section but writes:

```
scores_long |> summarize(mean_score = mean(score))
```

Identify **two problems**, write corrected code, and predict its output.

Problems: _____
Corrected code: _____

Output: _____
